Knowledge Graph e Graph Machine Learning

Analisi Comparativa e Implementazione con Oracle

Sandi Russo

Università degli Studi di Messina

Scienze Informatiche

Anno Accademico 2024/2025

Relatore: docente Antonio Celesti

Agenda

Il Dataset: L'Ecosistema PharMeBINet

Piani di Test: Analytics e Machine Learning

Implementazione Pratica: Setup e Ingestione Dati

Zero-Difference Benchmark e Data Forensics

Bilancio del Progetto e Roadmap Futura

II Dataset: L'Ecosistema PharMeBINet

L'Evoluzione dei Knowledge Graph Biomedici

Il Punto di Partenza: Hetionet

Hetionet (het.io)

- Storico Knowledge Graph biomedico.
- Ottimo punto di partenza teorico, ma limitato.
- Struttura obsoleta e instabile nell'importazione su DB moderni.

L'Evoluzione: PharMeBINet

The heterogeneous pharmacological network

- Fonde Hetionet con dati eterogenei moderni.
- *Data Engineering*: parsing, mapping di identificativi complessi e rimozione di sub-grafi ridondanti.

Riferimento Scientifico

Königs C., Friedrichs M., Dietrich T. *The heterogeneous pharmacological medical biochemical network PharMeBINet*. Scientific Data. 2022;9(1): 393.

Anatomia di PharMeBINet

Dati Generali del Grafo

Nodi: 5.831.471

Relazioni: 22.783.080

80 Labels — 277 Edge Types

Integrazione Dati Massiva

PharMeBINet non è un singolo file, ma aggrega e armonizza oltre **40 database biomedici**, tra cui:

- **DrugBank** (Farmacologia)
- **ClinVar & dbSNP** (Variazioni)
- **Gene Ontology** (Ontologie)
- **SIDER** (Effetti Collaterali)
- **DisGeNET** (Malattie)

Il Valore Aggiunto: Complessità e Multi-Modalità

Molto più di un semplice grafo sociale

La maggior parte dei tutorial sul Graph ML utilizza dataset "giocattolo" e omogenei (es. reti di citazioni come Cora o Citeseer). PharMeBINet offre una rete **eterogenea e multi-modale**.

Perché questa complessità è essenziale per i test?

- Permette di testare la capacità dei database di gestire query complesse (*multi-hop*) tra entità di natura profondamente diversa (es. *Gene* $\rightarrow$ *Proteina* $\rightarrow$ *Malattia* $\rightarrow$ *Farmaco*).
- Obbliga i modelli di Machine Learning a calcolare *Node Embeddings* che tengano conto non solo della topologia pura, ma del *tipo* di arco che collega i nodi.

Potenziale per il Graph Machine Learning

Le **Graph Neural Networks** applicate a questa struttura specifica sbloccano casi d'uso critici per la bioinformatica:

- **Link Prediction (Riposizionamento Farmaci)**: Se la GNN apprende il pattern topologico per cui un "Farmaco A" cura la "Malattia B", può prevedere archi futuri suggerendo cure inedite.
- **Previsione Effetti Collaterali**: Analizzando reti di interazione tra composti (`drug-drug interactions`), è possibile prevedere tossicità non documentate.
- **Community Detection**: Raggruppare entità in modo non supervisionato per scoprire pattern biologici nascosti, come network di geni co-espressi che partecipano alla stessa via metabolica.

Un banco di prova "Real-World"

La sfida dei dati reali

I dataset accademici standard sono perfettamente puliti. L'aggregazione di PharMeBINet porta con sé tutte le "sporcizie" tipiche degli scenari *Enterprise*.

Perché sceglierlo come Benchmark?

- Mettere sotto stress i motori di importazione (Neo4j APOC vs Oracle `SQL*Loader`).
- Testare la tolleranza ai difetti dei parser (caratteri invisibili, `\r`, EOF errati).
- Valutare come i diversi database (RDBMS vs Nativi) gestiscono le violazioni topologiche ("Dangling edges").

Piani di Test: Analytics e Machine Learning

Obiettivi di Testing: Graph Analytics

Analisi Topologica del Grafo

Centrality (Centralità)

- *Algoritmi*: PageRank, Betweenness Centrality.
- *Obiettivo*: Trovare i geni più influenti o i farmaci "hub".

Community Detection

- *Algoritmi*: Louvain, Weakly Connected Components.
- *Obiettivo*: Raggruppare entità biologiche con funzioni simili.

Path Finding

- *Algoritmi*: Shortest Path, A*.
- *Obiettivo*: Trovare la catena più breve tra farmaco e sintomo.

Obiettivi di Testing: Graph Machine Learning

Task di ML su Grafi

Node Embeddings

- Trasformare nodi in vettori preservando la struttura della rete.

Node Classification

- Classificare entità sconosciute basandosi sui loro "vicini".

Link Prediction

- **Task critico**: Prevedere collegamenti mancanti (es. cure potenziali).

Unsupervised Learning

- Clustering dei vettori per scoprire nuove tassonomie biomediche.

Implementazione Pratica: Setup e Ingestione Dati

Setup dell'Ambiente e Strumentazione

Isolamento e Sviluppo

- OS: Ubuntu tramite **WSL**
- Isolamento: `venv` Python
- Credenziali sicure via `.env`

Stack Librerie

- **oracledb**: Driver DB ufficiale
- **oml4py & pypgx**: ML & Grafi Oracle
- **neo4j**: Driver container

Inizializzazione Ambiente

```
# Creazione e attivazione venv
python3 -m venv .venv
source .venv/bin/activate

# Installazione dipendenze core
pip install oracledb neo4j python-dotenv pandas
      numpy jupyter oml4py pypgx
```

Inizializzazione di Neo4j

Importazione del Dataset PharMeBINet

I dati originali pre-processati (file `.dump`) sono stati caricati in un container temporaneo per popolare l'istanza definitiva.

```
# 1. Assegnazione permessi per il mount dei volumi Docker
sudo chown -R $USER:$USER $HOME/neo4j

# 2. Importazione del dump tramite container temporaneo
docker run --rm \
  -v $HOME/neo4j/data:/data \
  -v $HOME/neo4j/import:/import \
  -e NEO4J_ACCEPT_LICENSE_AGREEMENT=yes \
  neo4j:5-enterprise \
  neo4j-admin database load neo4j --from-path=/import --overwrite-destination=true

# 3. Avvio del container ufficiale
docker start neo4j-gds
```

L'Ostacolo della Versione Always Free

Il Limite di Oracle

- Limite **hard-coded di 12 GB**.
- TSV originali: oltre **30 GB**.
- Impossibile caricare tutto il DB relazionale in Oracle.

Data Reduction Pipeline

Estrazione chirurgica (Bash/AWK) per estrarre un sub-grafo chiuso di circa 8 GB senza rischiare di avere nodi "orfani".

Pipeline di Riduzione su TSV

```
# 1. Isoliamo i primi 8 milioni di archi
head -n 8000001 edges.tsv > edges_medium.tsv

# 2. Estraiamo gli ID univoci e ritagliamo i nodi corrispondenti dal file sorgente
awk -F'\t' 'NR>1 {print $3; print $4}' edges_medium.tsv | sort -u > valid_nodes.txt
awk -F'\t' 'NR==FNR {v[$1]; next} FNR==1 || ($1 in v)' valid_nodes.txt nodes.tsv > nodes_medium.tsv
```

Architettura e Importazione su Oracle DB

Preparazione Spazio

Allocazione di un `BIGFILE TABLESPACE` auto-espandibile per accomodare il traffico.

Scelta Ingegneristica

Nessuna Foreign Key in fase di creazione. Validare 8+ milioni di archi in inserimento avrebbe causato I/O insostenibile.

Travaso Alta Velocità (External Tables)

```
-- Lettura diretta dai TSV su disco (tramite ORACLE_LOADER)
-- e inserimento massivo bypassando i log (Direct Path Load)
INSERT /*+ APPEND */ INTO pharme_edges (relationship_id, type, start_id, end_id)
SELECT relationship_id, type, start_id, end_id
FROM ext_pharme_edges;
COMMIT;
```

Il Limite dei 12GB e la Soluzione ML-Oriented

Il Crash per i Campi Testuali

Il campo `properties` (CLOB) conteneva JSON testuali massicci, decuplicando lo spazio di archiviazione per via dell'overhead di Oracle e facendo scattare il blocco dei 12GB nonostante la riduzione a 8 milioni di archi.

La Soluzione orientata al Machine Learning

- Le **Graph Neural Networks** apprendono prevalentemente dalla **Topologia** e dalle **Labels**, non dal testo libero.
- **Azione:** Eliminazione dei CLOB testuali in fase di importazione, mantenendo solo ID strutturali, Label semantiche e Target relazionali.

Risultato del Caricamento: 5.831.471 Nodi e **22.783.080** Archi integrati.

Creazione del Property Graph Logico

Da Tabelle Relazionali a Grafo (SQL/PGQ)

I vincoli logici e topologici, omessi fisicamente nelle tabelle relazionali per performance, vengono imposti rigorosamente nel costrutto del Grafo Logico su Oracle PGX.

```sql
CREATE PROPERTY GRAPH pharme_graph
  VERTEX TABLES (
    pharme_nodes KEY (node_id)
      PROPERTIES (node_id, labels, name, identifier)
  )
  EDGE TABLES (
    pharme_edges KEY (relationship_id)
      SOURCE KEY (start_id) REFERENCES pharme_nodes (node_id)
      DESTINATION KEY (end_id) REFERENCES pharme_nodes (node_id)
      PROPERTIES (relationship_id, type)
  );
```

Zero-Difference Benchmark e Data Forensics

Allineamento Dati (Zero-Difference Benchmark)

Garantire la Validità Scientifica

Per un benchmark ML equo, Oracle e Neo4j devono operare sugli **stessi identici byte**. I dati snelliti di Oracle sono stati esportati in file CSV puri (`sqlplus SET MARKUP CSV`) per sovrascrivere l'istanza originale di Neo4j.

Risoluzione Problemi di Importazione (Neo4j APOC)

- **Pulizia Header:** Header CSV sporco rimosso via bash (`sed`).
- **Deadlock Multi-thread:** L'importazione massiva di archi con `parallel: true` causava eccezioni *ExclusiveLock* sui nodi centrali (hub).
- **Fix Applicato:** Fallback all'esecuzione thread-safe (`parallel: false`) sacrificando la velocità per garantire l'integrità transazionale.

Data Forensics: Indagine sulle Discrepanze

Il Nodo Fantasma (+1)

Problema: Neo4j riportava 1 nodo in più di Oracle.

Causa: Il parser APOC ha interpretato l'EOF come riga, generando un nodo generico `BaseNode` vuoto.

Fix: `MATCH (n:BaseNode) WHERE size(labels(n)) = 1 DETACH DELETE n;`

Gli Archi Orfani (-8.592)

Problema: Neo4j ha scartato 8.592 archi effettivamente presenti nel CSV.

Causa: Presenza di Carriage Return (`\r`) negli ID esportati, che impedivano il pattern-matching topologico di Neo4j.

Filosofia RDBMS vs Native Graph: La Prova del Nove

Comportamento Fisico vs Logico

- **Oracle DB (Fisico)**: Salva le righe tollerando inconsistenze (`\r`) e "dangling edges", agendo come pura tabella relazionale.
- **Neo4j (Nativo)**: Pulisce i caratteri e scarta gli archi orfani, non ammettendo violazioni strutturali della rete.

L'attivazione del Motore Graph (PGX)

Interrogando il Grafo Logico (`pharme_graph`) tramite SQL/PGQ, Oracle **filtra il rumore**:

```
SELECT COUNT(*) FROM GRAPH_TABLE ( pharme_graph MATCH ()-[r]->() ...  )
```

Risultato per entrambi: **22.783.080** archi validi.

Il motore PGX di Oracle applica lo stesso rigoroso filtro topologico del motore nativo di Neo4j.

Bilancio del Progetto e Roadmap Futura

Considerazioni Finali: Il Data Engineering

Obiettivo Infrastrutturale Raggiunto

Nonostante i pesanti limiti della versione *Free Tier*, è stato allestito con successo un ambiente di testing **Zero-Difference** su un dataset biomedico massivo (~23M relazioni).

Lezioni Apprese:

- L'inserimento massivo RDBMS richiede il bypass temporaneo dei vincoli (*No FK, Direct Path Load*).
- Nel Graph ML, la **topologia** conta più del testo descrittivo: eliminare JSON pesanti ha salvato il progetto senza compromettere i task predittivi.
- Il motore logico di Oracle (PGX) converte perfettamente l'anarchia relazionale nel rigore topologico nativo tipico di Neo4j.

Considerazioni Finali: Oracle vs Neo4j

Il Trade-off di Oracle

L'approccio di Oracle richiede una **curva di apprendimento molto più ripida** rispetto a Neo4j. Mantenere l'allineamento tra tabelle relazionali e astrazione logica (PGX) raddoppia la complessità del setup iniziale.

Il Vantaggio Strategico

Il compromesso è giustificato: la versione *Always Free* sblocca algoritmi **Enterprise-grade** e librerie native di ML (`OML4Py`) che in ecosistemi concorrenti richiederebbero licenze commerciali costose.

Prospettive: Benchmark e Algoritmi

Le fondamenta ingegneristiche gettate in questa fase permettono di passare all'esecuzione del **Graph Machine Learning** puro:

- **Benchmark Computazionale**: Esecuzione di *PageRank* e *Louvain* su 22 milioni di archi per confrontare i tempi di inferenza e il consumo RAM tra il motore PGX in-memory e Neo4j GDS.
- **Graph Neural Networks Avanzate**: Applicazione dei modelli *GraphSAGE* e *Graph Attention Networks (GAT)* tramite l'interfaccia Python `PyPGX`.
- **Link Prediction Biomedica**: Costruire un task di test reale per prevedere probabili iterazioni farmaco-malattia non presenti nel dataset.

Lavori Futuri: Task Predittivi con KG Embeddings

Node Classification tramite Embeddings

Utilizzo di tecniche di generazione di Knowledge Graph (KG) Embeddings per mappare i nodi in uno spazio vettoriale a bassa dimensionalità.

- **Caso d'Uso (PharMeBINet)**: Prevedere la categoria di appartenenza di un nuovo composto o inferire la funzione biologica di un gene non annotato, basandosi sulla sua posizione nello spazio latente.

Link Prediction basata su Relazioni

Generazione di embeddings specifici non solo per i nodi, ma anche per le **relazioni** (archi), al fine di calcolare la probabilità di esistenza di collegamenti mancanti.

- **Caso d'Uso (PharMeBINet)**: Riposizionamento dei farmaci (*Drug Repurposing*) prevedendo nuovi archi `TREATS` tra farmaci esistenti e malattie, o scoperta di interazioni avverse inedite.

Lavori Futuri: Tecniche di Embedding Relazionale

Per implementare i task di Link Prediction e Node Classification, si prevede di testare e confrontare i seguenti modelli di **Knowledge Graph Embedding**:

→ TransE (Translational Embeddings)

- Interpreta le relazioni come *traslazioni* nello spazio vettoriale ($h + r \approx t$).
- **Vantaggio**: Intuitivo ed efficiente, ideale per modellare gerarchie (*is-a*) o pattern 1-a-1.

⛶ DistMult

- Modello bilineare basato sulla fattorizzazione tensoriale con matrici diagonali.
- **Vantaggio**: Cattura in modo eccellente le interazioni *simmetriche* (es. interazioni proteina-proteina).

X^1 ComplEx (Complex Embeddings)

- Estende DistMult operando nel dominio dei **numeri complessi**.
- **Vantaggio**: Modella efficacemente le relazioni *asimmetriche* (es. un gene che regola un altro gene, ma non viceversa), dinamica fondamentale in biologia.

Risorse e Riferimenti

Oracle Database & Graph:

- Property Graph Docs: `docs.oracle.com/.../property-graph/`
- OML4Py: `docs.oracle.com/.../oml4py/`
- Graph Studio: `docs.oracle.com/.../graph-studio...`

Ecosistema & Alternative:

- PharMeBINet Dataset: `pharmebi.net`
- Neo4j: `graphacademy.neo4j.com`
- ArangoDB: `arangodb.com/docs/`

Librerie Machine Learning:

- PyTorch Geometric: `pytorch-geometric.readthedocs.io`
- DGL: `dgl.ai`

Grazie per l'attenzione!